

转转科技前端面试

一面 (技术基础) · 1-12

1. 谈谈你对 CSS 盒模型的理解，以及 box-sizing 的作用。
2. JavaScript 的基本数据类型有哪些？如何准确判断一个变量的类型？
3. 描述一下 JavaScript 的原型链机制。
4. 什么是闭包？它在实际开发中有什么应用场景？
5. 说一下 JavaScript 的事件循环（Event Loop）机制。
6. Promise 有哪几种状态？Promise.all 和 Promise.race 有什么区别？
7. 手写代码：实现一个深拷贝函数。
8. 手写代码：实现函数防抖（debounce）和节流（throttle）。
9. 浏览器缓存机制中的强缓存和协商缓存有什么区别？
10. 常见的 HTTP 状态码有哪些？分别代表什么含义？
11. 如何实现数组去重？请列举几种方法。
12. 算法题：给定一个只包括 '<code>'></code>

二面 (深入原理与项目) · 1-12

1. 谈谈 React Fiber 架构解决了什么问题？它的原理是什么？
2. Vue 3 的响应式原理与 Vue 2 有什么区别？
3. 谈谈你对虚拟 DOM (Virtual DOM) 的理解，它一定比直接操作 DOM 快吗？
4. Webpack 的构建流程是怎样的？你做过哪些构建优化？
5. 你们项目里是如何做前端性能优化的？
6. 什么是跨域？常见的解决方案有哪些？
7. 谈谈前端安全，XSS 和 CSRF 分别是什么？如何防范？
8. 在 Hybrid 开发中，JSBridge 的实现原理是什么？
9. 如果页面有一个万级数据的长列表，你会如何优化渲染？
10. 你们项目中的状态管理是如何选型的？Redux 和 Pinia 有什么区别？
11. 谈谈你对前端工程化的理解，Git 工作流是怎麼样的？

12. 如何监控前端页面的运行时错误？

三面（架构与综合能力）。1-6

1. 如果让你从零设计一个前端监控系统，你会考虑哪些模块？
2. 微前端架构解决了什么问题？你了解哪些实现方案？
3. 在团队中，你是如何推动技术规范或组件库建设的？
4. 面对一个老旧且复杂的项目，你会如何进行技术重构或迁移？

一面（技术基础）

1. 谈谈你对 CSS 盒模型的理解，以及 box-sizing 的作用。

难度：★

考点：CSS 布局基础

答案要点：

盒模型是浏览器渲染引擎根据标准将所有 HTML 元素表示为矩形盒子的机制，由内容、内边距、边框和外边距组成。

- 标准盒模型：width 仅包含 content，不包含 padding 和 border。
- 怪异盒模型：width 包含 content、padding 和 border。
- box-sizing 属性：content-box 对应标准模式，border-box 对应怪异模式。
- 实际应用：在响应式布局中，通常推荐使用 border-box 以简化尺寸计算。

常见坑：

- 忽略外边距重叠（Margin Collapsing）的情况。
- 混淆 padding 是否会撑开容器宽度。

追问：

- 如何触发 BFC？BFC 解决了什么问题？

2. JavaScript 的基本数据类型有哪些？如何准确判断一个变量的类型？

难度：★

考点：JS 基础类型

答案要点：

JS 包含 7 种原始类型和 1 种引用类型，判断类型需要根据场景选择 typeof、instanceof 或 Object.prototype.toString。

- 原始类型：String, Number, Boolean, Null, Undefined, Symbol, BigInt。
- 引用类型：Object（包括 Array, Function, Date 等）。
- typeof：适合判断原始类型，但判断 null 会返回 object。

- `Object.prototype.toString.call()`：最准确的方法，能识别内置对象类型。

常见坑：

- `typeof null` 为什么是 `object`。
- `instanceof` 无法跨 `iframe` 判断类型。

追问：

- 为什么 `0.1 + 0.2 !== 0.3`？如何解决？

3. 描述一下 JavaScript 的原型链机制。

难度：★★

考点：原型与继承

答案要点：

原型链是 JS 实现对象间属性共享和继承的机制，每个对象都有一个指向其构造函数原型的内部链接。

- 核心概念：每个对象都有 **proto**，每个函数都有 `prototype`。
- 查找过程：访问属性时，若当前对象没有，则沿 **proto** 向上查找，直到 `null`。
- 终点：原型链的顶端是 `Object.prototype.proto`，即为 `null`。
- 构造函数：`new` 操作符会将实例的 **proto** 指向构造函数的 `prototype`。

常见坑：

- 混淆 **proto** 和 `prototype` 的使用场景。
- 修改原型对象对已有实例的影响。

追问：

- 如何实现一个标准的寄生组合式继承？

4. 什么是闭包？它在实际开发中有什么应用场景？

难度：★★

考点：作用域与内存

答案要点：

闭包是指有权访问另一个函数作用域中变量的函数，本质是函数与其词法环境的引用捆绑。

- 形成条件：函数嵌套且内部函数引用了外部函数的变量。
- 应用场景：封装私有变量、柯里化、防抖节流、模块化模式。
- 内存影响：闭包会导致外部函数的变量常驻内存，不当使用可能引起内存泄漏。
- 生命周期：只要内部函数还在被引用，闭包中的变量就不会被垃圾回收。

常见坑：

- 在循环中使用 `var` 定义回调函数导致的闭包陷阱。
- 忘记手动解除不再需要的闭包引用。

追问：

- 闭包会导致内存泄漏吗？如何排查？

5. 说一下 JavaScript 的事件循环（Event Loop）机制。

难度：★★

考点：异步执行机制

答案要点：

事件循环是 JS 处理非阻塞 I/O 的核心机制，通过任务队列协调同步代码、微任务和宏任务的执行。

- 执行顺序：先执行同步代码（调用栈），再清空微任务队列，最后取出一个宏任务执行。
- 微任务：Promise.then、MutationObserver、process.nextTick。
- 宏任务：setTimeout、setInterval、setImmediate、I/O 操作。
- 渲染时机：浏览器通常在微任务清空后、下一个宏任务开始前尝试重新渲染。

常见坑：

- 错误认为 setTimeout 的延迟时间是精确的。
- 混淆 async/await 中 await 之后的代码执行顺序。

追问：

- 如果在微任务中不断产生新的微任务，会发生什么？

6. Promise 有哪几种状态？Promise.all 和 Promise.race 有什么区别？

难度：★★

考点：异步编程 API

答案要点：

Promise 是异步编程的一种解决方案，具有三种状态且状态转换不可逆。

- 三种状态：Pending（进行中）、Fulfilled（已成功）、Rejected（已失败）。
- Promise.all：所有 Promise 都成功才返回成功结果数组，只要有一个失败就立即报错。
- Promise.race：返回最快改变状态的那个 Promise 的结果，无论成功还是失败。
- 状态特性：一旦从 Pending 变为其他状态，就不会再变。

常见坑：

- Promise.all 传入空数组会立即同步返回成功。
- 忘记在 Promise 链末尾添加 catch 处理错误。

追问：

- 如何实现一个 Promise.allSettled？

7. 手写代码：实现一个深拷贝函数。

难度：★★

考点：递归与引用处理

答案要点：

深拷贝需要递归遍历对象属性，并处理循环引用以及特殊对象类型。

- 基础实现：使用递归处理 Object 和 Array。
- 循环引用：使用 WeakMap 存储已拷贝的对象，防止死循环。
- 类型处理：考虑 Date、RegExp、Map、Set 等特殊类型的克隆。
- 性能考量：避免过度递归导致栈溢出。

代码块

```
1  function deepClone(obj, map = new WeakMap()) {
2    if (obj === null || typeof obj !== 'object') return obj;
3    if (map.has(obj)) return map.get(obj);
4    let target = Array.isArray(obj) ? [] : {};
5    map.set(obj, target);
6    for (let key in obj) {
7      if (obj.hasOwnProperty(key)) {
8        target[key] = deepClone(obj[key], map);
9      }
10   }
11   return target;
12 }
```

常见坑：

- 使用 JSON.parse(JSON.stringify()) 丢失函数、Symbol 和 undefined。
- 忽略了原型链上的属性。

8. 手写代码：实现函数防抖（debounce）和节流（throttle）。

难度：★★

考点：高阶函数应用

答案要点：

防抖和节流都是为了限制函数执行频率，但触发逻辑不同。

- 防抖：在事件触发 n 秒后才执行，若 n 秒内再次触发则重新计时。
- 节流：在规定时间内只执行一次，无论触发多少次。
- 防抖场景：搜索框输入校验、窗口大小调整。
- 节流场景：滚动加载、鼠标移动跟踪。

代码块

```
1  function debounce(fn, delay) {
2    let timer = null;
```

```
3     return function(...args) {
4         if (timer) clearTimeout(timer);
5         timer = setTimeout(() => fn.apply(this, args), delay);
6     }
7 }
```

常见坑：

- 丢失 this 指向。
- 无法传递参数。

9. 浏览器缓存机制中的强缓存和协商缓存有什么区别？

难度：★★

考点：网络协议与性能

答案要点：

浏览器缓存通过 HTTP 响应头控制，旨在减少重复请求，提升加载速度。

- 强缓存：直接从本地读取，不发请求。由 Expires 和 Cache-Control 控制。
- 协商缓存：向服务器发请求验证资源是否过期。由 Last-Modified/If-Modified-Since 和 ETag/If-None-Match 控制。
- 状态码：强缓存返回 200 (from disk/memory cache)，协商缓存命中返回 304。
- 优先级：Cache-Control 优先级高于 Expires，ETag 优先级高于 Last-Modified。

常见坑：

- 误以为 304 也是强缓存。
- 忽略了 Ctrl+F5 强制刷新会跳过缓存。

追问：

- 为什么 ETag 比 Last-Modified 更可靠？

10. 常见的 HTTP 状态码有哪些？分别代表什么含义？

难度：★

考点：网络基础

答案要点：

HTTP 状态码由三位数字组成，首位数字定义了响应的类别。

- 2xx（成功）：200 OK，204 No Content。
- 3xx（重定向）：301 永久重定向，302 临时重定向，304 未修改。
- 4xx（客户端错误）：400 参数错误，401 未授权，403 禁止访问，404 未找到。
- 5xx（服务器错误）：500 服务器内部错误，502 网关错误，503 服务不可用。

常见坑：

- 混淆 301 和 302 对 SEO 的影响。
- 误用 401 和 403。

11. 如何实现数组去重？请列举几种方法。

难度：★

考点：数组操作

答案要点：

数组去重可以利用 ES6 新特性或传统的遍历比较来实现。

- Set: [...new Set(array)], 最简洁，但无法去重引用类型。
- Map/对象键值对：利用 key 的唯一性，效率高。
- filter + indexOf：通过判断当前索引是否为第一次出现的索引。
- 双重循环：最原始的方法，性能较差 ($O(n^2)$)。

常见坑：

- 无法处理 NaN 的去重 (indexOf 无法识别 NaN)。
- 引用类型 (如对象) 的去重逻辑。

12. 算法题：给定一个只包括 '(', ')', '{', '}', '[', ']' 的字符串，判断字符串是否有效。

难度：★★

考点：栈的应用

答案要点：

使用栈数据结构来匹配括号，遇到左括号入栈，遇到右括号与栈顶元素匹配。

- 匹配逻辑：右括号必须与最近的左括号匹配。
- 结束条件：遍历完成后栈必须为空。
- 边界处理：字符串长度为奇数直接返回 false。
- 映射关系：使用 Map 存储括号对，提高代码可读性。

常见坑：

- 栈为空时尝试弹出元素。
- 遍历结束后忘记检查栈是否为空。

二面（深入原理与项目）

1. 谈谈 React Fiber 架构解决了什么问题？它的原理是什么？

难度：★★★

考点：框架核心原理

答案要点：

Fiber 是 React 16 引入的新的协调引擎，主要解决了大型应用在更新时导致的页面卡顿问题。

- 核心问题：旧版 Stack Reconciler 递归更新不可中断，长期占用主线程导致掉帧。
- 任务拆分：将更新过程拆分为多个小单元（Fiber），支持暂停、恢复和优先级调度。
- 双缓存技术：在内存中构建 WorkInProgress Tree，完成后一次性提交到真实 DOM。
- 调度阶段：分为 Reconcile（可中断）和 Commit（不可中断）两个阶段。

常见坑：

- 误认为 Fiber 提升了渲染速度（实际上是提升了响应性）。
- 忽略了解生命周期在 Fiber 架构下的变化（如 `componentWillMount` 被废弃）。

追问：

- `requestIdleCallback` 在 Fiber 中起到了什么作用？

2. Vue 3 的响应式原理与 Vue 2 有什么区别？

难度：★★★

考点：响应式系统对比

答案要点：

Vue 3 弃用了 `Object.defineProperty`，改用 ES6 的 Proxy 来实现响应式系统。

- Proxy 优势：可以直接监听对象属性的添加和删除，支持数组索引和长度的修改。
- 性能优化：Proxy 是延迟监听，只有访问到深层属性时才递归处理，初始化更快。
- 依赖收集：Vue 3 使用 Map 和 Set 建立 `target -> key -> effect` 的映射关系。
- 兼容性：Proxy 无法在 IE11 中 polyfill，这是 Vue 3 不支持 IE 的主要原因。

常见坑：

- 混淆 `ref` 和 `reactive` 的使用场景。
- 忽略了解构 `reactive` 对象会丢失响应性的问题。

追问：

- 为什么 Vue 3 还需要 `ref`？

3. 谈谈你对虚拟 DOM（Virtual DOM）的理解，它一定比直接操作 DOM 快吗？

难度：★★

考点：渲染性能

答案要点：

虚拟 DOM 是对真实 DOM 的一层抽象，通过 JS 对象描述 UI 结构。

- 核心价值：跨平台能力、函数式编程、通过 Diff 算法减少不必要的 DOM 操作。
- 性能误区：在极致优化的原生操作面前，虚拟 DOM 并不更快，它提供的是性能下限的保障。

- Diff 算法：通过同层比较、Key 值复用等策略将复杂度从 $O(n^3)$ 降至 $O(n)$ 。
- 批量更新：收集多次状态变更，一次性应用到真实 DOM。

常见坑：

- 认为虚拟 DOM 是为了解决性能问题的唯一方案。
- 不理解 Key 在 Diff 过程中的真正作用。

4. Webpack 的构建流程是怎样的？你做过哪些构建优化？

难度：★★★

考点：工程化实践

答案要点：

Webpack 构建是一个串行的过程，从入口开始递归解析依赖，通过 Loader 转换文件，最后通过 Plugin 输出 Bundle。

- 核心流程：初始化参数 -> 开始编译 -> 确认入口 -> 编译模块 -> 完成模块编译 -> 输出资源。
- 速度优化：持久化缓存（cache）、多进程打包（thread-loader）、缩小搜索范围（include/exclude）。
- 体积优化：Tree Shaking、代码分割（SplitChunks）、压缩代码（Terser）、图片压缩。
- 辅助工具：webpack-bundle-analyzer 分析包体积。

常见坑：

- 滥用 Loader 导致构建变慢。
- 忽略了 SourceMap 在生产环境对体积的影响。

追问：

- Loader 和 Plugin 的区别是什么？

5. 你们项目里是如何做前端性能优化的？

难度：★★★

考点：综合实战能力

答案要点：

性能优化是一个系统工程，需要从加载性能、渲染性能和运行时性能三个维度入手。

- 加载优化：静态资源 CDN、Gzip 压缩、路由懒加载、图片懒加载/WebP。
- 渲染优化：减少回流重绘、使用 CSS3 硬件加速、防抖节流、虚拟列表。
- 指标监控：关注 FP、FCP、LCP、CLS 等核心 Web 指标。
- 策略工具：利用 Lighthouse 进行评估，使用性能分析面板定位长任务。

常见坑：

- 盲目优化，没有数据支撑。
- 优化了非关键渲染路径，导致投入产出比低。

6. 什么是跨域？常见的解决方案有哪些？

难度：★★

考点：浏览器安全

答案要点：

跨域是由浏览器的同源策略引起的，限制了不同源（协议、域名、端口）之间的资源交互。

- CORS：最主流方案，服务器设置 Access-Control-Allow-Origin 等响应头。
- Proxy：开发环境下通过 Webpack DevServer 代理，生产环境下通过 Nginx 反向代理。
- JSONP：利用 script 标签不受同源策略限制的特性，仅支持 GET 请求。
- PostMessage：适用于不同窗口或 iframe 之间的跨域通信。

常见坑：

- 混淆“请求发不出去”和“响应被拦截”。
- 忽略了 CORS 预检请求（OPTIONS）的触发条件。

7. 谈谈前端安全，XSS 和 CSRF 分别是什么？如何防范？

难度：★★

考点：安全意识

答案要点：

前端安全主要防范恶意脚本注入和身份冒用攻击。

- XSS（跨站脚本攻击）：攻击者在页面注入恶意脚本。防范：输入过滤、输出转义、使用 CSP。
- CSRF（跨站请求伪造）：冒用用户身份发送非法请求。防范：使用 Token 验证、SameSite Cookie 属性、验证 Referer。
- 区别：XSS 是获取页面控制权，CSRF 是利用用户的登录状态。
- 现代框架：React/Vue 默认会对插值内容进行转义，能防范大部分基础 XSS。

常见坑：

- 认为使用了 HTTPS 就能防范 XSS/CSRF。
- 忽略了 innerHTML 带来的安全风险。

8. 在 Hybrid 开发中，JSBridge 的实现原理是什么？

难度：★★★

考点：混合开发技术

答案要点：

JSBridge 建立了 Native 和 Web 之间的双向通信通道，使 H5 能够调用原生能力。

- JS 调用 Native：拦截 URL Schema、注入 API（addJavascriptInterface）。
- Native 调用 JS：直接执行 JS 代码字符串（evaluateJavascript）。
- 异步回调：JS 调用时传入 callbackId，Native 处理完后通过该 ID 触发 JS 回调。

- 封装库：通常会封装一套 SDK，屏蔽底层通信细节。

常见坑：

- URL Schema 长度限制问题。
- 注入 API 在不同系统版本下的安全性问题。

9. 如果页面有一个万级数据的长列表，你会如何优化渲染？

难度：★★★

考点：复杂场景处理

答案要点：

处理长列表的核心思路是“按需渲染”，避免一次性创建大量 DOM 节点。

- 虚拟列表：只渲染可视区域内的节点，滚动时动态计算偏移量并替换内容。
- 时间分片：利用 requestAnimationFrame 将大量 DOM 创建任务拆分到多帧执行。
- 列表项优化：固定高度以简化计算，使用 key 提升 Diff 效率。
- 交互优化：滚动过程中停止高耗时操作，使用骨架屏提升感知速度。

常见坑：

- 滚动过快导致的白屏闪烁。
- 动态高度列表的计算复杂度。

10. 你们项目中的状态管理是如何选型的？Redux 和 Pinia 有什么区别？

难度：★★

考点：架构选型

答案要点：

状态管理选型应基于项目规模和团队习惯，目标是实现数据流的可预测和可维护。

- Redux：流程严谨（Action -> Reducer -> Store），适合大型复杂项目，中间件生态丰富。
- Pinia：Vue 3 推荐，API 简洁，支持多 Store 结构，类型推导友好，无 Mutations。
- 核心区别：Redux 强调不可变性和单一数据源，Pinia 更加轻量且符合 Composition API 风格。
- 适用场景：简单项目可用 Context 或组合式函数，复杂业务再引入状态库。

常见坑：

- 过度使用状态管理，将所有局部状态都存入全局 Store。
- 在 Reducer 中执行异步操作。

11. 谈谈你对前端工程化的理解，Git 工作流是怎么样？

难度：★★

考点：团队协作与规范

答案要点：

前端工程化涵盖了从开发、构建、测试到部署的全生命周期，旨在提升开发效率和代码质量。

- 规范化：ESLint、Prettier、Commitlint 确保代码风格一致。
- 自动化：CI/CD 流程，通过 Jenkins 或 GitHub Actions 自动执行测试和部署。
- Git 工作流：通常采用 Git Flow 或主干开发模式，包含 feature、develop、release、hotfix 分支。
- 模块化：通过 ESM 或 CommonJS 实现代码解耦。

常见坑：

- 缺少代码审查（Code Review）环节。
- 配置文件在团队间不统一。

12. 如何监控前端页面的运行时错误？

难度：★★

考点：稳定性保障

答案要点：

前端监控分为错误监控、性能监控和行为监控，错误监控是保障稳定性的第一道防线。

- 捕获方式：window.onerror 捕获同步错误，unhandledrejection 捕获 Promise 异常。
- 资源加载错误：通过 window.addEventListener('error', callback, true) 在捕获阶段获取。
- 框架异常：Vue.config.errorHandler 或 React Error Boundaries。
- 上报策略：使用 navigator.sendBeacon 确保页面卸载时也能成功上报，并进行采样和去重。

常见坑：

- 跨域脚本错误只显示 "Script error"。
- 上报请求过多导致后端压力过大。

三面（架构与综合能力）

1. 如果让你从零设计一个前端监控系统，你会考虑哪些模块？

难度：★★★

考点：系统设计能力

答案要点：

设计监控系统需要闭环考虑数据采集、传输、存储、分析和报警五个环节。

- 采集模块：封装 SDK，自动收集异常、性能指标（LCP/FID）、用户行为轨迹。
- 传输模块：支持批量上报、压缩数据、离线存储（IndexedDB）以及断网续传。
- 处理模块：后端进行数据清洗、SourceMap 解析还原代码位置、设备信息解析。
- 可视化与报警：提供看板展示趋势，设置阈值通过钉钉/邮件实时告警。

- 业务价值：通过数据驱动性能优化，快速定位线上故障，提升用户留存。

常见坑：

- 忽略了监控 SDK 本身对页面性能的影响。
- 存储压力过大，没有做好数据降级和清理策略。

2. 微前端架构解决了什么问题？你了解哪些实现方案？

难度：★★★

考点：架构视野

答案要点：

微前端旨在将大型单体应用拆分为多个独立交付的微应用，解决技术栈老旧、构建慢、协作难的问题。

- 核心价值：技术栈无关、独立开发部署、增量升级、应用隔离。
- 常见方案：iframe（隔离最强但体验差）、qiankun（基于 single-spa，生态成熟）、Module Federation（Webpack 5 原生支持）。
- 隔离机制：样式隔离（Shadow DOM/Scoped CSS）、JS 隔离（Proxy Sandbox）。
- 通信机制：CustomEvent、全局 Store 或 URL 传参。

常见坑：

- 拆分粒度过细导致维护成本激增。
- 忽略了公共依赖抽离和版本冲突问题。

3. 在团队中，你是如何推动技术规范或组件库建设的？

难度：★★★

考点：软技能与影响力

答案要点：

推动基建需要从痛点出发，通过标准制定、工具支撑和持续迭代来落地。

- 发现痛点：识别重复造轮子、样式不统一或代码质量差等实际问题。
- 制定标准：产出组件开发规范、文档说明，并与 UI/产品达成共识。
- 落地工具：利用 Storybook 展示组件，通过 npm 私有库发布，集成到脚手架中。
- 持续演进：收集业务方反馈，定期重构，保持组件的灵活性和稳定性。

常见坑：

- 闭门造车，设计的组件不符合业务场景。
- 缺乏文档和示例，导致推广困难。

4. 面对一个老旧且复杂的项目，你会如何进行技术重构或迁移？

难度：★★★

考点：复杂问题解决

答案要点：

重构应遵循“小步快跑、不影响业务”的原则，避免推倒重来带来的风险。

- 现状评估：理清业务逻辑、依赖关系，识别核心链路和高风险区域。
- 制定计划：先建立自动化测试保障逻辑正确，采用微前端或多页应用方式逐步替换。
- 灰度发布：通过功能开关（Feature Toggle）或流量切分，先在小范围验证。
- 监控跟进：实时监控重构后的报错率和性能指标，确保平滑过渡。

常见坑：

- 盲目追求新技术而忽略了业务连续性。
- 缺少文档记录，导致重构后出现隐蔽 Bug。

5. 聊聊你在过去一年中遇到的最有挑战的技术问题，你是如何解决的？

难度：★★★

考点：复盘与总结

答案要点：

这是一个开放性问题，重点考察候选人的问题分析能力、技术深度和解决问题的韧性。

- 背景描述：简洁交代业务背景和技术环境。
- 问题定义：明确问题的现象、影响范围以及定位过程中的难点。
- 解决方案：对比多种方案的优劣，说明最终选择的理由及实施步骤。
- 总结沉淀：问题解决后的效果（数据支撑），以及对团队或后续开发的启示。

常见坑：

- 描述过于琐碎，没有重点。
- 解决方案显得过于平庸，体现不出技术含量。

6. 你对前端未来的发展趋势怎么看？最近在关注哪些新技术？

难度：★★

考点：技术热情与视野

答案要点：

考察候选人是否保持学习习惯，以及对行业动态的敏锐度。

- 趋势方向：如 WebAssembly 扩展浏览器边界、AI 与前端结合（Copilot/低代码）、边缘计算等。
- 关注点：可以提到 Rust 在前端工具链中的应用（如 SWC, Turbopack）、Serverless 带来的全栈化。
- 学习方法：通过阅读源码、参与开源、关注技术社区或撰写博客来保持进步。
- 落地思考：新技术如何解决当前业务中的实际问题，而非为了用而用。

常见坑：

- 只提名词不理解原理。
- 表现出对技术发展的焦虑而非热情。